It's easiest to think of it as a **deterministic data-processing pipeline wrapped in a flexible multi-agent orchestration layer**.
LLMs are used for planning, validation and reflection, but they never touch the raw numbers – Python/pandas do all of the work.

🔁 Overall run-loop

Every execution is a closed-loop coordinated by the **Orchestrator**:

1. **Planner** – produces an ordered list (or graph) of actions.
2. **Executor** – carries out each action, writing results to short-term memory.
3. **Validator** – runs deterministic checks (and optionally an LLM review).
4. **Reflection** – inspects outputs and suggests next steps (retry, re-plan, investigate).
5. **Orchestrator** – enforces retries, idempotency, circuit-breaking, and logs everything.

"LLMs propose & critique; Python is the source of truth."

📦 Core components & their roles

Component	Responsible for
PlannerAgent / LLMPlanner	Translate objectives into steps. Can be deterministic or LLM-assisted.
ExecutorAgent	Implements the actual data transformations (load, assign_quarter, aggregate, churn, validate, reflect).
ValidatorAgent / LLMValidator	Checks totals, churn sanity, etc. Optionally asks an LLM to review outputs.
ReflectionAgent	Scores results and returns suggestions ("investigate lost clients", "retry aggregation", etc.).
MemoryManager	Tracks short-term step outputs, compresses summaries to long-term memory, maintains idempotency and a tiny semantic index.
CircuitBreaker	Stops repeating failing steps after a threshold.
MetricsCollector	Simple in-process counters & timers written to metrics.json.

GraphRunner	Executes a dependency graph, honouring depends_on and allowing parallelism.
SemanticMemory	Deterministic vector store used by agents/LLMs for retrieval.

🔧 Tools package (tools.py)
Deterministic helpers called by executors:

- DataLoader – handles CSV/Excel loading, normalises wide-format sheets.
- assign_quarter, Aggregator, ChurnCalculator – compute quarterly and client-level aggregates.
- Validator – sanity-checks and totals match.
- SemanticMemory – local, reproducible embedding store.
- MemoryManager – higher-level wrapper combining short/long/semantic memory.

🔄 Execution workflow in code

1. **Initialization**
   Orchestrator.__init__ wires up agents, optional per-role LLM clients, circuit breaker, memory, metrics, and idempotency state.
2. **Planning**
   - planner.plan() returns a list of dicts; LLMPlanner can override with an LLM result.
   - Example node:
     {
   - "id": "agg_q1",
   - "action": "aggregate_quarter_region",
   - "depends_on": ["load"],
   - "validation_checkpoint": "validate_quarterly"
   - }

3. **Graph execution**
   - GraphRunner.run() respects explicit depends_on and sequence order.
   - Ready nodes may run in parallel (bounded by max_workers).
   - Each node dispatches to Orchestrator._run_single_step().
4. **Single-step handling**
   - Idempotency key computed from step name + hashed context.
   - Circuit breaker checked.
   - Retries with exponential backoff (configurable max_retries, backoff_base).
   - Success logs, metrics, and memory updates are recorded.
   - Validation steps trigger ValidatorAgent; failures can invoke the LLM

validator.

5. **Post-run reflection & output**
    - Reflection suggestions gathered.
    - Metrics and logs are dumped to outputs.
    - Visualizations generated (churn trends, quarterly totals, lost-client plots).

🧠 Memory model

- **Short-term** – per-step summaries kept in memory during a run.
- **Long-term** – compressed summaries persisted to memory.json; idempotency flags, failure counts.
- **Semantic** – optional vector store (semantic_memory.json); deterministic embeddings support small retrievals for prompt context.

Memory managers are called throughout (store_step_summary, mark_executed, etc.); tests cover both plain and semantic variants.

🔁 Reliability & observability features

- **Retries & backoff** – every step wrapped with a retry loop.
- **Circuit breaker** – after a configurable number of failures, a step is "open" and future attempts abort.
- **Idempotency** – prevents repeated side-effects when a step is re-executed with the same input.
- **Metrics** – counters and timings recorded; persisted to JSON.
- **Logs** – per-step status, errors, attempts stored in self.log.
- **Tests** – numerous unit tests simulate failures, chunking, parallel runs, planner budgets, LLM clients, etc.

🧩 Multi-agent collaboration

The "multi-agent" aspect is largely architectural:

1. **Separation of concerns** – planning, executing, validating, reflecting, memory, LLM interaction are separate agents/objects.
2. **Pluggability** – LLMs can be swapped in per role (planner, validator, reflection).
3. **Deterministic fallbacks** – every LLM-based agent has a pure-Python alternative.
4. **GraphRunner** lets agents' actions be composed into a directed acyclic graph with dependencies.
5. **Semantic memory** is shared between agents for context.

Agents communicate via the context dict and the MemoryManager; the Orchestrator orchestrates, but the implementation of each step lives in its own class.

📦 Sample sequence (no LLM)

1. PlannerAgent.plan() ➜ [{"step":"load_data"},

{"step":"assign_quarter"}, ...]
2. Orchestrator runs load_data:
    - reads CSV/Excel,
    - stores df in context,
    - logs row count & revenue sum.
3. assign_quarter adds a “Quarter” column.
4. aggregate_quarter_region groups and writes a CSV.
5. client_quarterly pivots per-client revenue.
6. compute_churn compares successive quarters and dumps JSON.
7. validate checks totals match.
8. reflect inspects churn results and notes “significant_loss”.
9. Orchestrator persists metrics & visualizations, returns final result.

With LLMs enabled, the planner/validator/reflection may interject suggestions or even replan entire flows if the LLM advises.

✅ Workflow highlights from tests

- Chunking (with/without parallel workers) produces identical churn reports.
- GraphRunner correctly respects dependencies and parallelism.
- Circuit breaker trips after repeated failures.
- LLMPlanner respects token budgets and falls back when necessary.
- Semantic memory returns stable results.
- Orchestrator accepts per-role LLM clients and still runs normally.

🗃️ Visual/Output artifacts

The run writes to the outputs directory:

- quarterly_by_region.csv, client_quarterly_revenue.csv
- churn_report.json (plus timestamped variants)
- metrics.json, memory.json, semantic_memory.json
- Visualizations (.png, .html) for churn and revenue trends.

Final note

This architecture is intentionally **deterministic, testable, and transparent**. The multi-agent terminology describes a **modular design** where separate classes (“agents”) own discrete responsibilities and communicate via shared state under orchestration.

The workflow is a classic **Plan → Execute → Validate → Reflect → Act** loop, with control flow managed by the Orchestrator and optional LLMs providing higher-level reasoning within a tightly-scoped numerical system.